

1. Analisi

1.1 Descrizione e requisiti

Il software in fase di sviluppo prende il nome di "J-Navy" e si pone l'obiettivo di realizzare una versione moderna ed evoluta del classico gioco da tavolo della Battaglia Navale. L'applicazione è progettata per automatizzare le meccaniche base del gioco, gestendo le regole di posizionamento delle flotte e l'alternanza dei turni. Il giocatore sfiderà una CPU su una griglia di dimensioni 10x10, posizionando una flotta composta da navi di diverse dimensioni.

A differenza della versione tradizionale, "J-Navy" introduce elementi innovativi volti ad arricchire la componente strategica del gioco: i **Capitani** e le **Condizioni Atmosferiche**. Per "Capitano" si intende una figura di comando, scelta all'inizio della partita, dotata di abilità uniche che possono influenzare drasticamente lo scontro. Tali abilità variano dal supporto difensivo, all'attacco offensivo ad area, fino allo spionaggio tattico. L'utilizzo di queste abilità è soggetto a un *cooldown*, ossia un tempo di ricarica misurato in turni, che impedisce l'abuso dei poteri speciali. Per "Condizioni Atmosferiche" si intendono invece variabili ambientali generate dinamicamente dal sistema durante il corso della partita. Questi eventi atmosferici alterano temporaneamente le meccaniche di base, aggiungendo un ulteriore livello di imprevedibilità alla sfida.

Requisiti funzionali

- Il sistema deve permettere la configurazione iniziale della partita, consentendo al giocatore di scegliere il proprio Capitano e di impostare il livello di difficoltà del bot avversario.
- Il sistema deve gestire il posizionamento della flotta (sia manuale da parte dell'utente, sia generato casualmente dal sistema), validando le coordinate per impedire sovrapposizioni o posizionamenti fuori dai confini della griglia.
- Il sistema deve orchestrare l'alternanza dei turni di gioco tra l'utente e la CPU, determinando l'esito dei colpi e aggiornando lo stato delle flotte.
- Il sistema deve calcolare e scalare i tempi di ricarica (*cooldown*) delle abilità speciali dei capitani, permettendone l'uso solo quando disponibili.
- Il sistema deve poter variare dinamicamente le condizioni atmosferiche col passare dei turni, applicando eventuali malus (es. deviazione dei colpi) in caso di maltempo.
- Il sistema deve decretare la fine della partita e dichiarare il vincitore non appena tutte le navi di una delle due flotte vengono affondate.
- Il sistema deve permettere il salvataggio dello stato della partita in corso e il suo successivo caricamento.

Requisiti non funzionali

- **Usabilità (Responsività):** L'interfaccia grafica dovrà adattarsi dinamicamente alle diverse dimensioni e risoluzioni degli schermi, garantendo un'esperienza visiva ottimale e mantenendo le proporzioni della griglia di gioco.

- **Usabilità (Feedback):** Il sistema dovrà fornire un chiaro e immediato feedback visivo e sonoro per ogni azione saliente (es. animazioni e suoni per colpi a segno, colpi a vuoto, attivazione di abilità o cambio del meteo).
- **Prestazioni:** I tempi di decisione e di risposta della CPU durante il suo turno dovranno essere sufficientemente rapidi da non interrompere il flusso e il ritmo della partita.

1.2 Modello del Dominio

Il dominio applicativo di "J-Navy" è caratterizzato dallo scontro tattico a turni alterni tra due entità principali: i **Giocatori** (un Umano e un Bot). Ciascun giocatore è proprietario di una **Griglia** di gioco, la quale rappresenta il proprio campo di battaglia. Tale griglia è composta da un insieme di **Celle**, ciascuna dotata di un proprio stato interno fondamentale per tenere traccia degli eventi (come l'esito di un colpo subito o la presenza di un'imbarcazione).

All'interno della griglia, ogni giocatore schiera una **Flotta**, formata da un insieme di **Navi**. Ogni nave è a sua volta caratterizzata da:

- lo spazio occupato sulla griglia.
- un valore rappresentante la vita rimasta, che si riduce a ogni colpo incassato portando la nave all'affondamento una volta esaurito.

Le interazioni primarie tra le entità avvengono tramite i **Colpi**. Durante il proprio turno, un giocatore effettua un colpo che è determinato da:

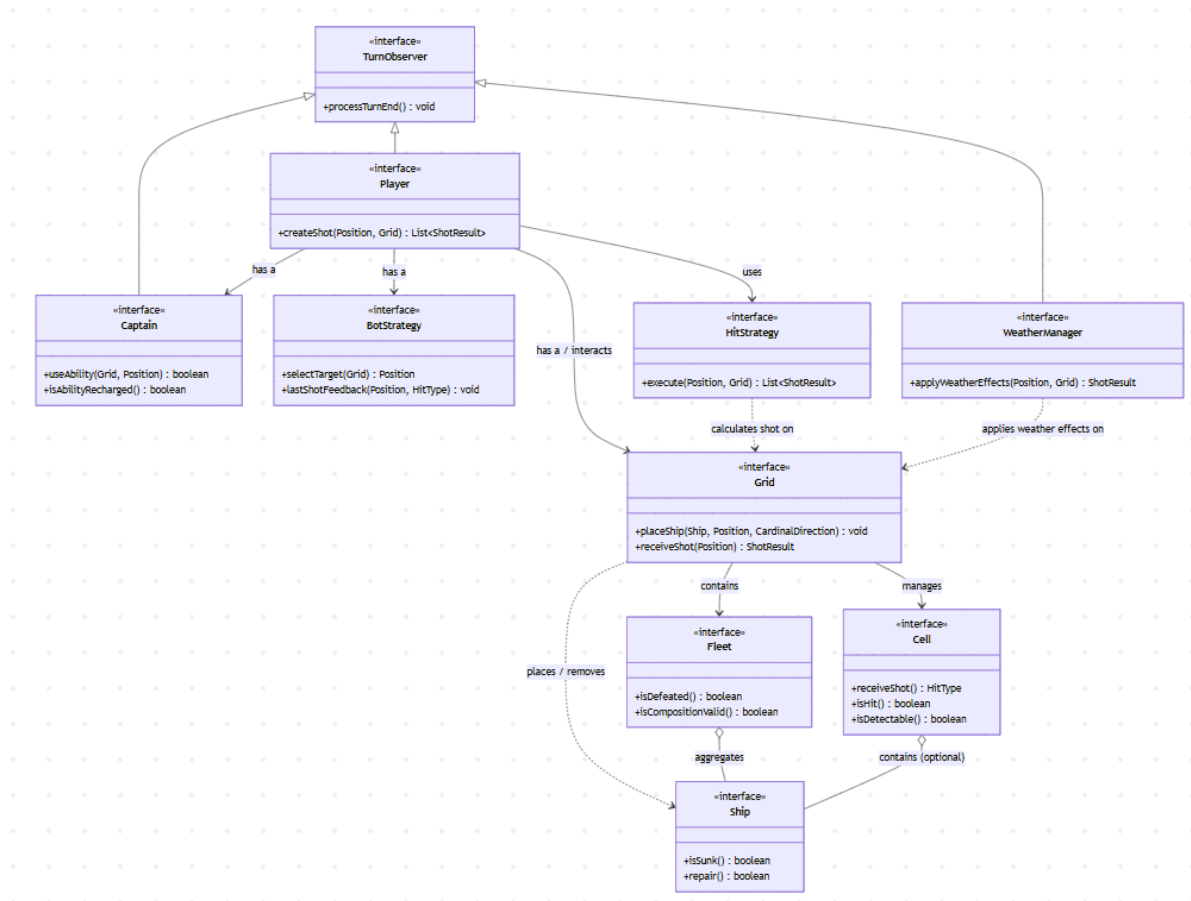
- una coordinata bersaglio sulla griglia avversaria.
- un esito (mancato, colpito, affondato), il quale altera in modo permanente lo stato della cella colpita e della nave eventualmente presente.

Una differenza fondamentale tra le due tipologie di giocatori risiede nelle meccaniche di gioco a loro disposizione:

- Il **Giocatore Umano** è rappresentato dalla figura del **Capitano**, che guida la flotta. Ogni capitano ha a disposizione un'**Abilità Speciale** unica. L'uso di tali abilità è strettamente vincolato da un **Tempo di Ricarica (cooldown)**, un contatore basato sul susseguirsi dei turni che ne impedisce l'uso ripetuto.
- Il **Bot (Nemico)** è invece caratterizzato da una **Strategia** che ne regola il comportamento.

L'intero scontro è inoltre immerso in un ecosistema ambientale governato dalle **Condizioni Atmosferiche**. Il meteo è un'entità dinamica che:

- evolve ciclicamente col passare dei turni.
- ha il potere di interferire con le azioni di base, alterando, ad esempio, la precisione dei colpi lanciati.



2. Design

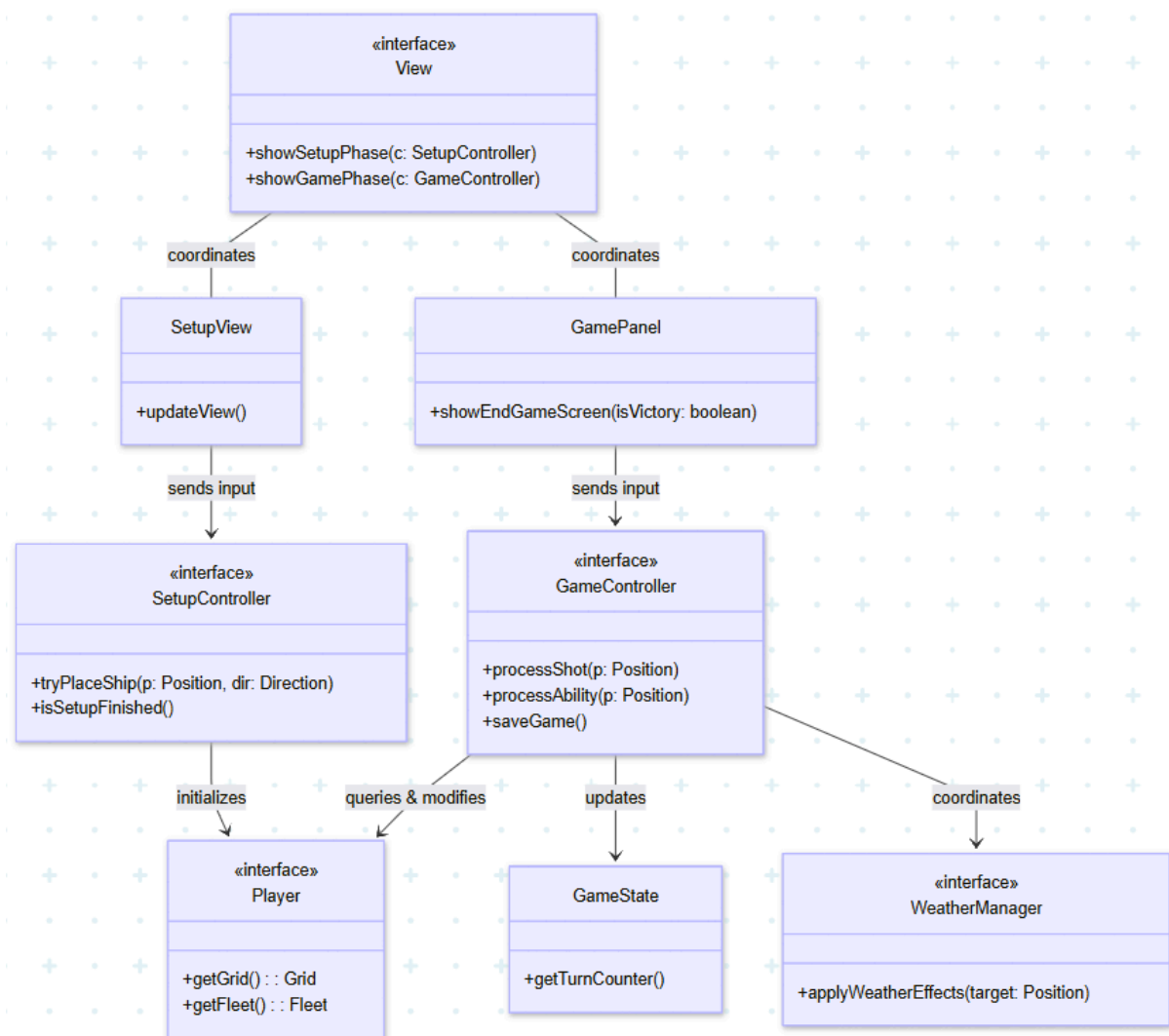
2.1 Architettura

L'architettura di J-Navy segue il pattern architetturale Model-View-Controller (MVC). Questo pattern è stato adottato per garantire un basso accoppiamento fra le parti, disaccoppiando in modo netto la logica di dominio dalla sua rappresentazione visiva e dalla gestione dell'interazione con l'utente. Al fine di evitare la creazione di un'unica macro-interfaccia, il sistema definisce dei punti d'ingresso specifici per ciascun componente architetturale.

Il ruolo di controller in J-Navy è definito principalmente dall'interfaccia GameController che, affiancata da SetupController, svolge i compiti di regia: riceve gli input generati dall'utente, interroga e modifica lo stato di gioco e coordina l'aggiornamento della partita. La componente grafica del sistema è definita dall'interfaccia View, che funge da coordinatore per l'intera interfaccia utente. Il ruolo di View è quello di gestire unicamente la navigazione tra gli stati dell'applicazione. Il rendering di dettaglio e la cattura specifica degli eventi di input sono invece delegati a un insieme di sotto-viste specializzate per ogni fase di gioco (come i pannelli di setup e la dashboard di combattimento), le quali a loro volta comunicano con il rispettivo controller.

La logica di dominio risiede interamente nel Model, il quale è del tutto ignaro dell'esistenza della View e del Controller. Per evitare un accesso disordinato ai dati, i punti di accesso principali per questa componente sono incapsulati in interfacce di alto livello: Player funge da radice per accedere alle entità operative e spaziali del gioco (come la Griglia, la Flotta e il Capitano), mentre i gestori come il WeatherManager amministrano i mutamenti ambientali. La persistenza è invece delegata alla componente GameState, che fotografa lo stato della partita rendendolo serializzabile.

Grazie a questa rigida separazione tramite interfacce, l'architettura di J-Navy garantisce l'assoluta indipendenza della rappresentazione visiva: un'eventuale sostituzione in blocco della View richiederebbe unicamente la scrittura di nuove classi visive, senza causare alcuna singola modifica all'interno dei Controller e tantomeno all'interno del Modello.



2.2 Design dettagliato

2.2.1 Rocca

In questa sezione si approfondisce il design relativo alla gestione delle abilità speciali dei giocatori, illustrando le scelte architetturali adottate per favorire l'estensibilità del sistema e minimizzare la duplicazione del codice.

Gestione delle Abilità Speciali (Strategy Pattern)

Problema

All'interno del dominio di gioco, il giocatore umano deve poter variare le proprie abilità e meccaniche speciali in base alla figura scelta all'inizio della partita. Inserire la logica di tutte le possibili abilità (attacco ad area, cura, sonar, ecc.) direttamente all'interno della classe del giocatore avrebbe portato a un codice monolitico, scarsamente coeso e vincolato a continui costrutti condizionali (es. switch-case sul tipo di capitano). Inoltre, l'aggiunta di una nuova tipologia di abilità in futuri aggiornamenti avrebbe richiesto la modifica di classi centrali del modello.

Soluzione

Per risolvere il problema si è scelto di applicare il design **pattern Strategy**. L'interfaccia **Captain** rappresenta la **strategia astratta**, esponendo il contratto per l'uso dell'abilità tramite il metodo `useAbility()`. La classe **Human** agisce come **Context** del pattern: possiede un riferimento all'interfaccia **Captain** e delega a essa l'esecuzione dell'azione. Questa soluzione rispetta appieno il principio *Open-Closed*: l'aggiunta di un nuovo capitano con abilità inedite richiede esclusivamente la creazione di una nuova classe concreta, senza la necessità di alterare minimamente il codice della classe **Human**.

Gestione dei Cooldown e Riutilizzo del Codice (Template Method Pattern)

Problema

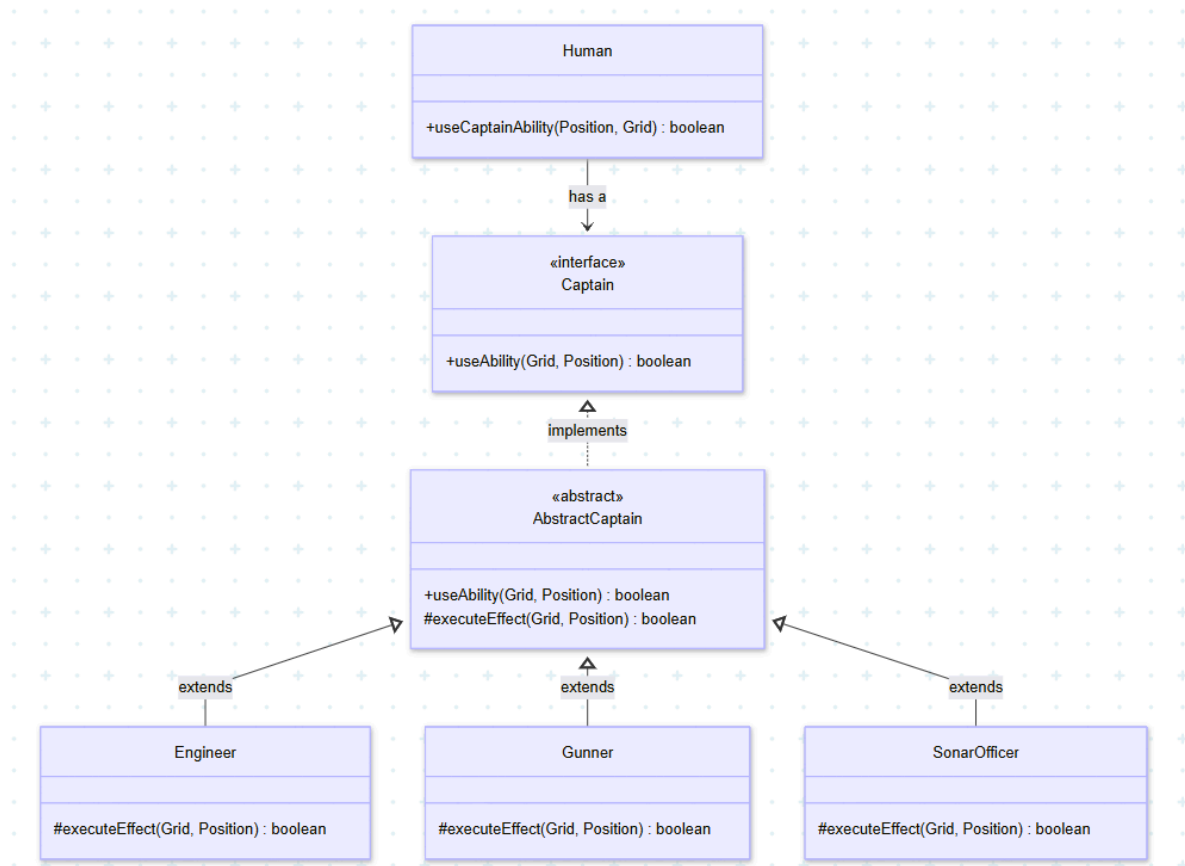
Durante la progettazione delle varie implementazioni di **Captain** (es. **Gunner**, **Engineer**, **SonarOfficer**), è emerso che tutte condividevano la medesima logica di validazione e di gestione temporale: prima di poter eseguire un'abilità, è sempre necessario verificare che il **cooldown** (tempo di ricarica) sia esaurito e che le coordinate bersaglio siano valide. Lasciare l'onere di questi controlli alle singole classi concrete avrebbe comportato una massiccia duplicazione del codice. Peggio ancora, avrebbe esposto il sistema a errori critici: un'implementazione distratta in una sottoclasse avrebbe potuto dimenticare di resettare il **cooldown** dopo l'uso, rompendo il bilanciamento del gioco.

Soluzione

Si è deciso di utilizzare il design pattern **Template Method**, introducendo la classe **AbstractCaptain** come livello intermedio nella gerarchia. Per garantire la massima sicurezza e coerenza del sistema, **tutti i metodi pubblici** ereditati dall'interfaccia sono stati intenzionalmente marcati come **final**. Questo approccio impedisce categoricamente

l'overriding da parte delle sottoclassi, blindando in un unico punto sicuro tutta la logica di validazione e la gestione temporale.

Il metodo **executeEffect()** è l'unico ad essere dichiarato come **protected e abstract**: esso rappresenta il "passo" dell'algoritmo delegato ai figli. In questo modo, classi concrete come Gunner o Engineer si limitano a definire esclusivamente *cosa* fa l'abilità, mentre la classe madre controlla rigorosamente il *quando* e il *come* essa possa essere attivata.



2.2.2 Antonellini

In questa sezione si illustrano le scelte architetturali adottate per la gestione degli Automated Player (Bot) e per lo sviluppo di un sistema modulare, flessibile e facilmente manutenibile, avvalendosi di design pattern consolidati per risolvere specifiche criticità di design.

Gestione degli Automated Player (Strategy Pattern)

Problema

Nel dominio applicativo, è necessario dotare il giocatore controllato dal computer (il Bot) di diversi comportamenti o "livelli di difficoltà" (in questo caso: Beginner, Pro, Sniper). Il problema principale consiste nel dover variare l'algoritmo di selezione del bersaglio e la reazione all'esito del colpo, senza dover modificare la classe principale del Bot a ogni

aggiunta o revisione di un comportamento. Inserire queste logiche tramite costrutti condizionali (come pesanti switch-case) all'interno del Bot avrebbe portato a un codice difficilmente estensibile.

Soluzione

Ho scelto di delegare le logiche decisionali a classi esterne applicando il **design pattern Strategy**. La classe **Bot** agisce come **Context** del pattern: possiede un riferimento all'interfaccia **BotStrategy** e delega a essa l'esecuzione dell'azione. L'interfaccia rappresenta la strategia astratta, esponendo il contratto tramite metodi come **selectTarget()**. Un dettaglio architetturale rilevante è la definizione del metodo **lastShotFeedback()** come default (con corpo vuoto) all'interno dell'interfaccia: questo approccio evita di forzare i bot più semplici (come **BeginnerBot**) o di complicare inutilmente bot di difficoltà maggiore ma di più facile implementazione (come **SniperBot**) e a fornire implementazioni fittizie e inutili, lasciando l'onere dell'override solo alle intelligenze avanzate (come **ProBot**) che necessitano di apprendere dall'esito dei colpi. Questa soluzione rispetta appieno il principio Open-Closed: l'aggiunta di una nuova difficoltà richiede solo la creazione di una nuova classe concreta.

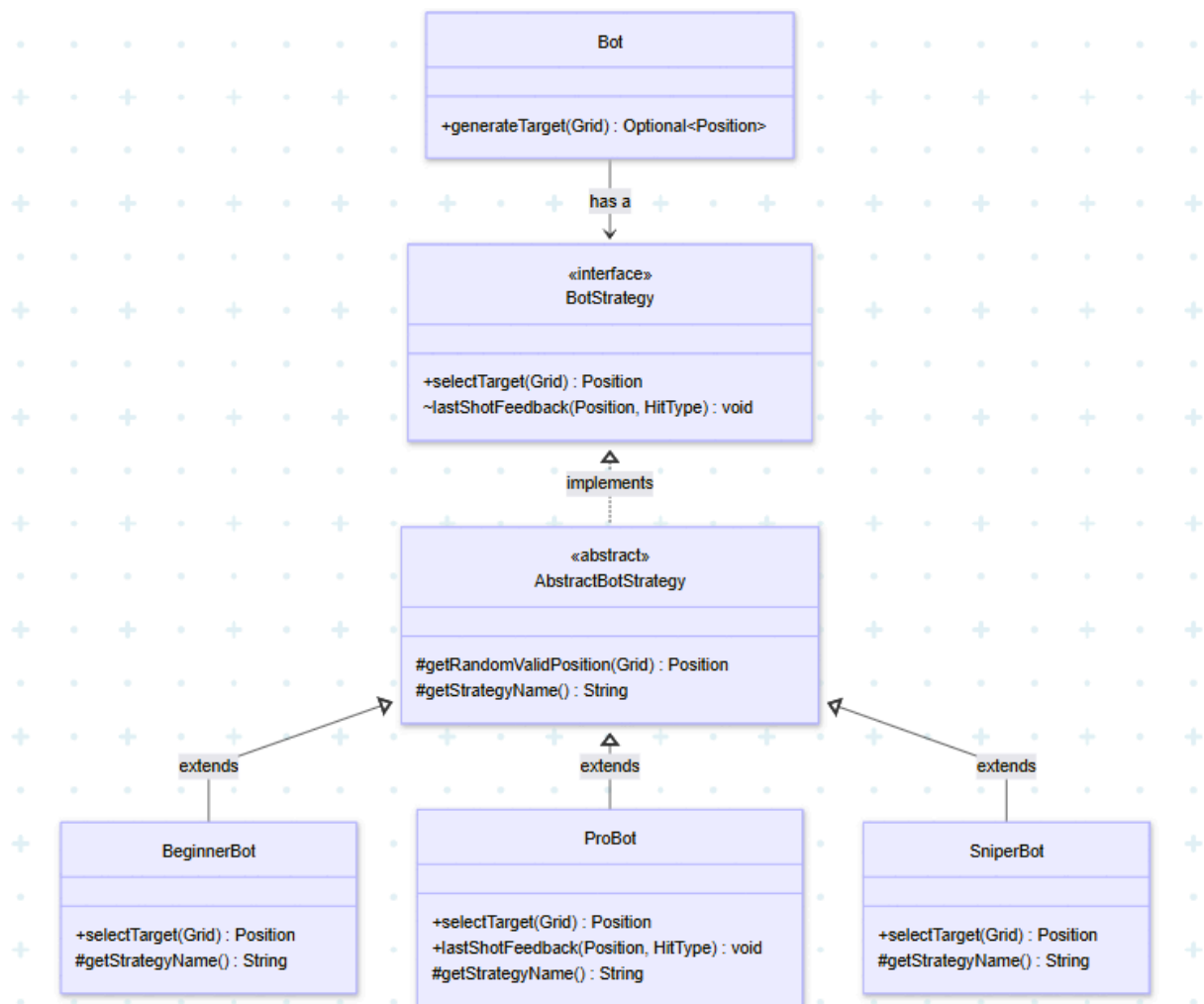
Minimizzazione della Duplicazione e Riutilizzo del Codice (Template Method Pattern)

Problema

Analizzando le implementazioni delle strategie concrete, è emersa la necessità di condividere porzioni di codice e utility ricorrenti, come l'estrazione di coordinate casuali valide, e di definire uniformemente l'identificativo testuale della strategia. Lasciare l'implementazione interamente a carico di ogni singolo bot avrebbe generato un'evidente duplicazione del codice e costretto ogni classe a gestire autonomamente uno stato interno (come l'oggetto **Random** necessario per i calcoli), portando a un design poco coeso.

Soluzione

Per massimizzare il riutilizzo del codice (principio DRY), ho deciso di utilizzare il **design pattern Template Method**, introducendo la classe **AbstractBotStrategy** come livello intermedio nella gerarchia. Essa funge da **AbstractClass**, centralizzando lo stato condiviso e fornendo alle sottoclassi metodi protected di utilità generale (come `getRandomValidPosition()` e `getValidCellsList()`), pronti all'uso e non da riscrivere. Inoltre, la classe astratta definisce il Template Method **getStrategy()**, il quale orchestra lo scheletro per il recupero del nome delegando la specificità alla Primitive Operation protected abstract String **getStrategyName()**. In questo modo, le sottoclassi concrete si limitano a implementare esclusivamente il proprio dettaglio specifico, mentre la classe madre gestisce in modo sicuro e centralizzato l'infrastruttura condivisa..



2.2.3 Solaroli

In questa sezione si approfondiscono le scelte progettuali relative al 'core model' dell'applicazione, ovvero il sotto-sistema responsabile della gestione del campo di battaglia: **Grid, Fleet, Cell e Ship**. Si è posta una forte enfasi sulla sicurezza e sulla robustezza del codice, eliminando l'uso di riferimenti nulli e restringendo l'accesso agli stati interni del modello. A tal fine, si è fatto largo uso di composizione, delega delle responsabilità, e dell'incapsulamento protetto (ad esempio tramite l'uso di Optional e copie difensive).

Delega delle responsabilità e gestione sicura dell'assenza di stato (Cell e Ship)

Problema: All'interno della griglia di gioco, una cella può rappresentare una porzione di mare oppure può essere occupata da una nave. Il problema progettuale principale consisteva nel determinare come modellare questa dualità senza rischiare errori a runtime (come le NullPointerException) e, contemporaneamente, stabilire a chi assegnare la responsabilità di gestire i danni in caso di attacco.

Soluzione: Un'opzione di modellazione avrebbe potuto essere tramite ereditarietà; ma preferito utilizzare *composizione* e *delega*. L'interfaccia Cell mantiene al suo interno un riferimento alla nave che la occupa. Per gestire in modo robusto l'assenza di una nave, si è scelto di non restituire mai null all'esterno, ma di incapsulare il ritorno nel costrutto

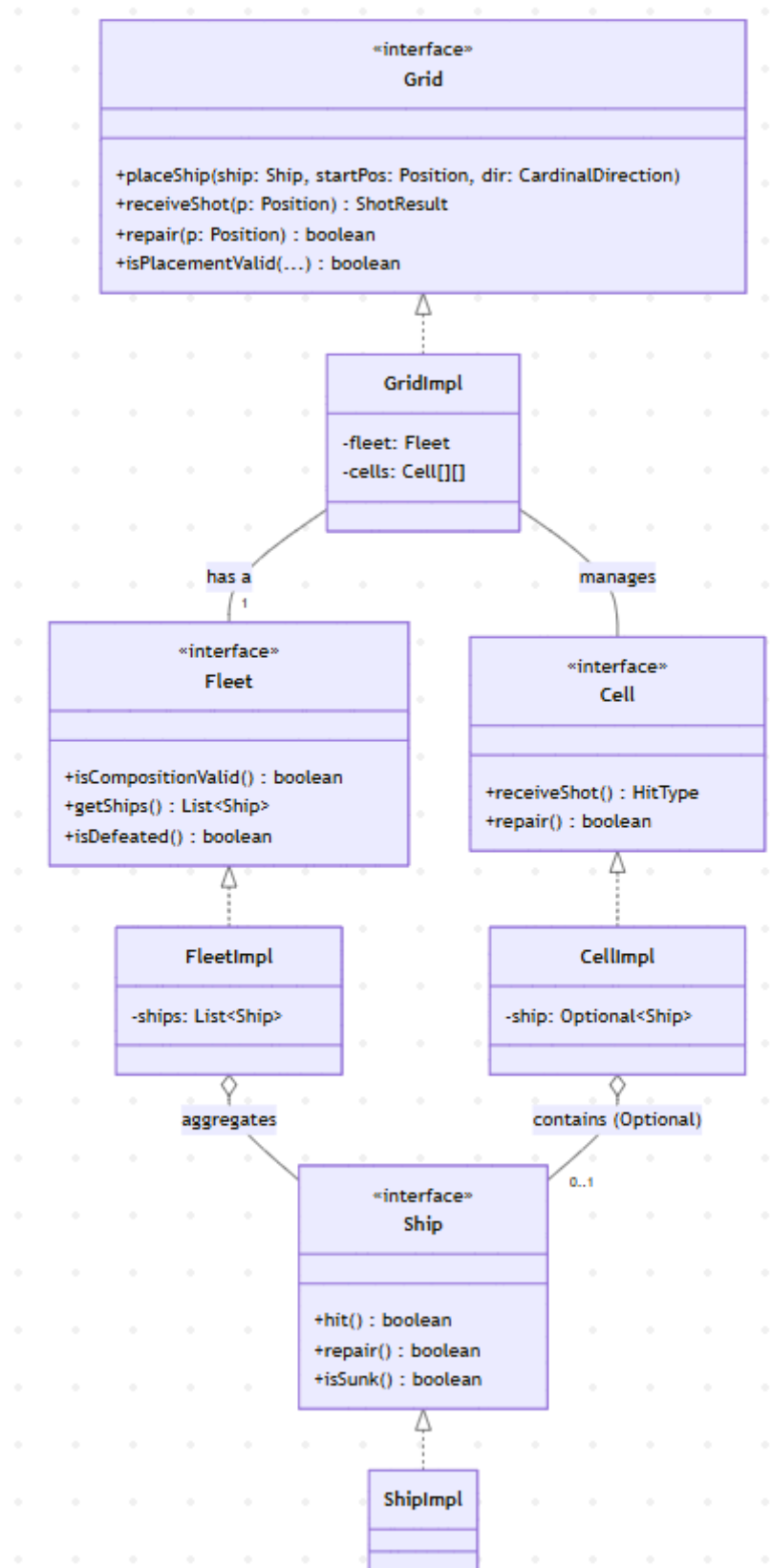
Optional<Ship>. Questo design brilla particolarmente nella gestione delle alterazioni di stato. Quando una cella subisce un attacco tramite `receiveShot()`, o al contrario viene riparata tramite `repair()`, la cella non altera mai direttamente lo stato interno della nave. Invece, delega l'azione chiamando i rispettivi metodi `hit()` o `repair()` dell'interfaccia `Ship`. In questo modo l'incapsulamento è massimizzato: la classe `Cell` si limita a gestire il proprio stato visivo/logico (sapere se è stata colpita o se è tornata integra), mentre la logica di calcolo dei danni, della salute e l'eventuale verifica di affondamento (`isSunk()`) restano di competenza esclusiva della classe `Ship`. Questo design rende estremamente facile introdurre in futuro nuove tipologie di ostacoli o navi con comportamenti di danno personalizzati, senza dover alterare la logica della cella.

Separazione delle responsabilità e validazione topologica (Grid e Fleet)

Problema: In un gioco navale esistono regole ferree riguardanti la composizione valida di una flotta (ad esempio: 1 nave da 5 celle, 1 nave da 4 celle, ecc.). Il problema consisteva nel decidere dove implementare il controllo di validità di queste regole. Implementare il conteggio e la validazione direttamente all'interno della griglia (`Grid`) avrebbe reso quest'ultima una classe troppo grande e complessa (violando il `Single Responsibility Principle`), mischiando la logica spaziale di posizionamento bidimensionale con le logiche di stato del giocatore.

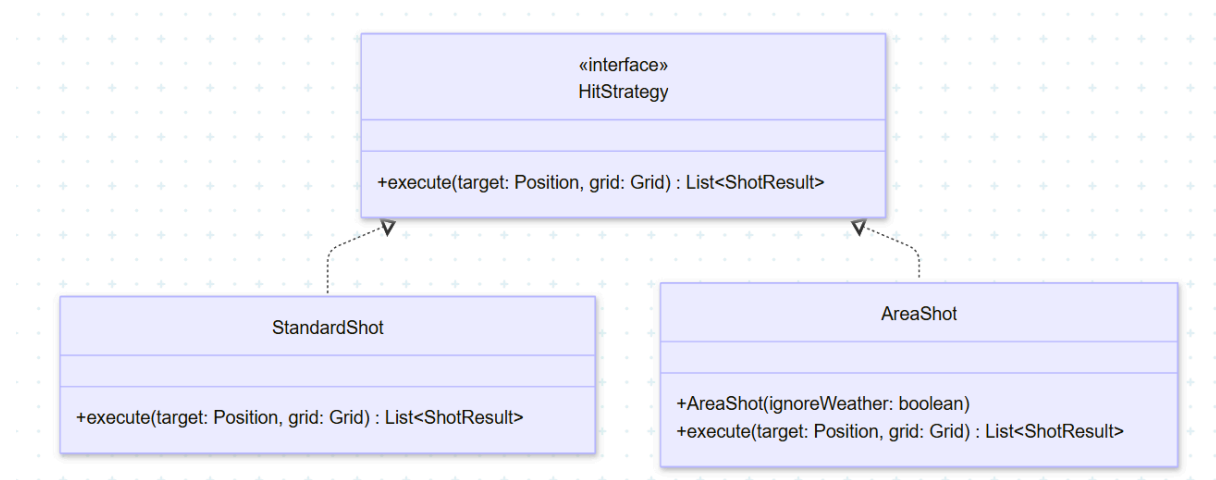
Soluzione: Ho deciso di separare i concetti estraendo la gestione della logica di composizione nell'interfaccia `Fleet`. La `Grid` funge da coordinatore del posizionamento: si occupa esclusivamente della validazione spaziale (verificando i confini e le collisioni tramite `isPlacementValid()`) e di assegnare i riferimenti della nave alle celle interessate.

Contestualmente, durante la fase di posizionamento (`placeShip()`), la griglia delega alla sua istanza interna di `Fleet` il compito di registrare la nave appena piazzata tramite l'invocazione diretta di `addShip()`. La classe `FleetImpl` funge così da raccogliitore logico e si fa carico di validare le regole del gioco tramite una mappa costante (`FLEET_COMPOSITION`) e i metodi `isCompositionValid()` e `isDefeated()`. Questo disaccoppiamento garantisce che un eventuale cambio delle regole di gioco sulla composizione della flotta non impatti minimamente la complessa logica bidimensionale della `Grid`. Inoltre, per prevenire malfunzionamenti o modifiche accidentali da parte di altre classi, la flotta implementa la copia difensiva: il metodo `getShips()` restituisce una `List.copyOf()`, proteggendo l'immutabilità della struttura interna ed evitando alterazioni di stato non previste.



2.2.4 Bandini

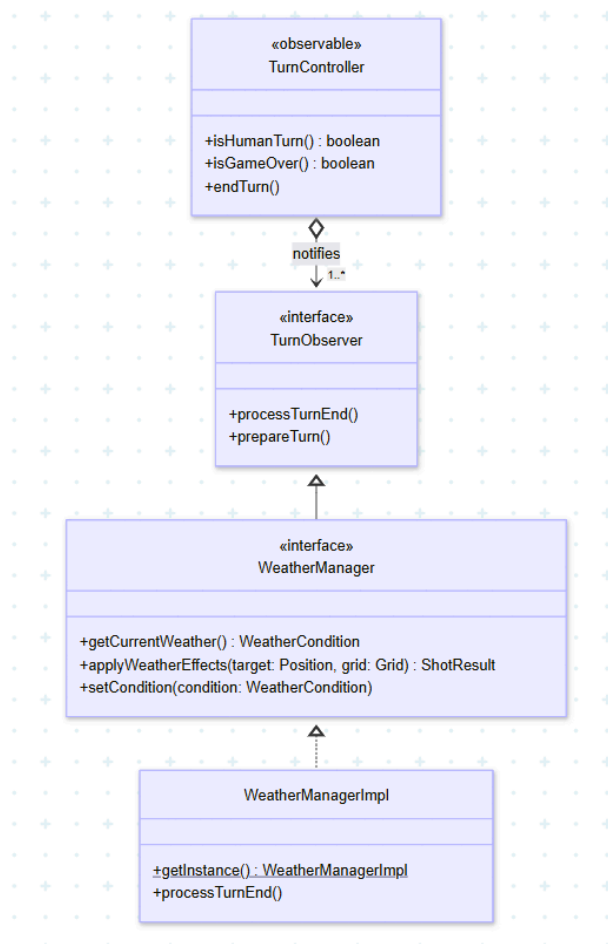
Gestione delle Abilità di Tiro (Pattern Strategy)



Problema: All'interno del dominio di gioco, i Capitani possono disporre di diverse abilità, tra cui di un'abilità di attacco (il gunner), che permette di usare un colpo ad area (`AreaShot`) invece del colpo standard (una cella singola). Inizialmente, il rischio architetturale maggiore era quello di concentrare la logica di calcolo dei bersagli all'interno delle classi dei giocatori o del controller, portando alla creazione di molti costrutti `if/else` o `switch` basati sul tipo di colpo. Questo avrebbe violato l'Open/Closed Principle, rendendo ogni futura aggiunta di nuove abilità soggetta a errori e richiedendo la modifica di codice già consolidato.

Soluzione: Per risolvere il problema abbiamo applicato il Pattern Strategy. Abbiamo creato un'interfaccia `HitStrategy` che espone il metodo `execute()`, spostando la logica dei singoli colpi in classi separate (`StandardShot` e `AreaShot`). In questo modo, "chi spara" non deve preoccuparsi di "come" il colpo colpisce la griglia: si limita a chiamare il metodo. Se in futuro volessimo aggiungere un nuovo tipo di sparo, ci basterà creare una nuova classe che implementa l'interfaccia, senza dover toccare e rischiare di rompere il codice che già funziona.

Gestione del Meteo Dinamico (Pattern Observer)



Problema: Una delle meccaniche principali del gioco prevede un sistema meteo dinamico, capace di alternare condizioni di visibilità perfetta (Sunny) a condizioni di nebbia (Fog) capaci di deviare i colpi. La sfida principale consisteva nel far evolvere lo stato del meteo in funzione del passare dei turni, senza violare però il Single Responsibility Principle. Il motore dei turni non avrebbe dovuto quindi possedere alcuna conoscenza diretta delle logiche atmosferiche.

Soluzione: La problematica è stata risolta adottando il Pattern Observer. La classe **TurnController** funge da Observable (Subject): la sua unica responsabilità, al termine di ogni turno, è aggiornare il contatore interno e lanciare una notifica generalizzata invocando il metodo `processTurnEnd()` su tutti gli elementi in ascolto. L'interfaccia **TurnObserver** funge da Observer. Il **WeatherManager** (e la sua implementazione concreta **WeatherManagerImpl**) implementa questa interfaccia, registrandosi come ascoltatore. Quando riceve la notifica di fine turno, il gestore del meteo calcola in totale autonomia se le condizioni atmosferiche debbano mutare. Questa architettura permette alle due classi di comunicare perfettamente, restando però indipendenti l'una dall'altra.

3. Sviluppo

3.1 Testing automatizzato

Per il testing automatico del progetto abbiamo utilizzato JUnit. Ci siamo concentrati sulle logiche centrali del Model e sulle meccaniche più complesse del gioco, per garantire robustezza ed evitare bug fastidiosi durante la partita. Tutti i test realizzati sono completamente automatizzati e riproducibili.

Ecco i componenti principali che abbiamo verificato:

- **GridTest** e **CellTest**: Verificano la corretta gestione del tabellone di gioco. Abbiamo testato che il posizionamento delle navi rispetti i limiti della mappa e prevenga collisioni. Inoltre, controllano rigidamente la logica degli spari (esito Acqua, Colpito, Affondato).
- **FleetTest** e **ShipTest**: Testano l'integrità della flotta e delle singole navi. Verificano che la flotta possa essere composta solo da un numero preciso di navi in base alla dimensione, e controllano il sistema di danno, di affondamento e le meccaniche di riparazione.
- **CaptainTest**: Si concentra sulle abilità speciali. Verifica che il sistema di cooldown (gestito dalla classe astratta) si ricarichi correttamente con il passare dei turni. Inoltre, testa individualmente gli effetti concreti dei capitani: la riparazione dell'Ingegnere, il colpo ad area 2x2 del Gunner e il raggio di scansione 3x3 del Sonar.
- **WeatherManagerTest**: Testa la solidità del sistema meteo. Oltre a verificare il reset del Singleton, controlla l'alternanza probabilistica Sole/Nebbia al superamento della soglia dei turni. Una parte cruciale del test verifica la matematica della deviazione dei colpi durante la Nebbia, assicurandosi che il colpo venga deviato di massimo una cella senza mai finire fuori dalla griglia (Edge Case) e senza colpire celle già rivelate.
- **BotTest**: Verifica il comportamento del bot. Abbiamo testato le tre diverse strategie: la validità dei bersagli casuali del BeginnerBot, il corretto cambio di stato del ProBot (da Hunting a Seeking fino al Destroying quando trova una nave) e infine la logica truccata dello SniperBot, accertandoci che miri alle navi con la giusta percentuale di successo.

3.2 Note di sviluppo

Rocca

1. Uso combinato di Optional e lambda:
<https://github.com/Roccia2005/OOP25-J-Navy/blob/8a31dc0b175eae455ea86c0b82df13c8e6a4982b/src/main/java/it/unibo/jnavy/controller/game/GameStateController.java#L109>
2. Utilizzo combinato di Stream e lambda:
<https://github.com/Roccia2005/OOP25-J-Navy/blob/8a31dc0b175eae455ea86c0b82df13c8e6a4982b/src/main/java/it/unibo/jnavy/model/captains/SonarOfficer.java#L40>
3. Utilizzo lambda in vari punti:

<https://github.com/Roccia2005/OOP25-J-Navy/blob/8a31dc0b175eae455ea86c0b82df13c8e6a4982b/src/main/java/it/unibo/jnavy/view/game/GameDashboardPanel.java#L46>

4. Utilizzo del pacchetto javax.imageio:

<https://github.com/Roccia2005/OOP25-J-Navy/blob/8a31dc0b175eae455ea86c0b82df13c8e6a4982b/src/main/java/it/unibo/jnavy/view/utilities/ImageLoader.java#L16>

Antonellini

1. Utilizzo di Optional per ottenere la posizione generata dall'Automated Player:

https://github.com/Roccia2005/Progetto_OOP/blob/b83761720b65c68b198c040bc459924e24c0e064/src/main/java/it/unibo/jnavy/model/player/Bot.java#L93

2. Utilizzo di Lambda e Method Reference per eseguire la StartView iniziale:

https://github.com/Roccia2005/Progetto_OOP/blob/b83761720b65c68b198c040bc459924e24c0e064/src/main/java/it/unibo/jnavy/view/ViewGUI.java#L90

3. Utilizzo di Functional Interface per determinare l'Automated Player selezionato:

https://github.com/Roccia2005/Progetto_OOP/blob/b83761720b65c68b198c040bc459924e24c0e064/src/main/java/it/unibo/jnavy/view/selection/BotSelectionPanel.java#L272

Solaroli

1. Uso combinato di Stream, Lambda e Optional:

<https://github.com/Roccia2005/OOP25-J-Navy/blob/b83761720b65c68b198c040bc459924e24c0e064/src/main/java/it/unibo/jnavy/model/grid/GridImpl.java#L170>

2. Programmazione funzionale e null-safety con Optional:

https://github.com/Roccia2005/Progetto_OOP/blob/b83761720b65c68b198c040bc459924e24c0e064/src/main/java/it/unibo/jnavy/model/grid/GridImpl.java#L110

3. Utilizzo di Stream:

https://github.com/Roccia2005/Progetto_OOP/blob/b83761720b65c68b198c040bc459924e24c0e064/src/main/java/it/unibo/jnavy/controller/setup/SetupControllerImpl.java#L181

4. Utilizzo del pacchetto javax.sound.sampled:

<https://github.com/Roccia2005/OOP25-J-Navy/blob/b83761720b65c68b198c040bc459924e24c0e064/src/main/java/it/unibo/jnavy/view/utilities/SoundManager.java#L18>

Bandini

1. Utilizzo di Record e Optional per definire l'esito dei colpi in modo sicuro e senza boilerplate:

https://github.com/Roccia2005/Progetto_OOP/blob/f10ea4146a7e0f4a07a65f57f6ec84e0edfabaa7/src/main/java/it/unibo/jnavy/model/utilities/ShotResult.java#L18

2. Utilizzo di Lambda Expression e interfacce funzionali per gestire animazioni e fine turno:

https://github.com/Roccia2005/Progetto_OOP/blob/f10ea4146a7e0f4a07a65f57f6ec84e0edfabaa7/src/main/java/it/unibo/jnavy/view/components/EffectsPanel.java#L68

3. Utilizzo di AtomicInteger per rendere thread-safe il contatore dei turni:

https://github.com/Roccia2005/Progetto_OOP/blob/f10ea4146a7e0f4a07a65f57f6ec84e0edfabaa7/src/main/java/it/unibo/jnavy/model/weather/WeatherManagerImpl.java#L116

4. Utilizzo di Graphics2D e RenderingHints per disegnare dinamicamente l'interfaccia meteo:
https://github.com/Roccia2005/Progetto_OOP/blob/b83761720b65c68b198c040bc459924e24c0e064/src/main/java/it/unibo/jnavy/view/components/weather/WeatherNotificationOverlay.java#L84-L88
5. Utilizzo di javax.swing.Timer per orchestrare le animazioni visive e le trasparenze:
https://github.com/Roccia2005/Progetto_OOP/blob/b83761720b65c68b198c040bc459924e24c0e064/src/main/java/it/unibo/jnavy/view/components/weather/WeatherNotificationOverlay.java#L48-L56

4. Commenti finali

4.1 Autovalutazione e lavori futuri

Rocca

Sono molto orgoglioso del lavoro svolto, essendomi occupato principalmente della progettazione del sistema dei Capitani, delle loro abilità speciali e della logica di controllo centrale del gioco (GameController e GameState).

Il punto di forza del mio contributo risiede nell'architettura estensibile creata per le abilità. L'utilizzo combinato dei pattern Strategy e Template Method ha garantito un codice pulito, sicuro e privo di duplicazioni, blindando la logica dei *cooldown* e rendendo banale l'aggiunta di nuovi personaggi. Il mio ruolo nel gruppo è stato quello di "ponte" tra le meccaniche base del modello e le feature avanzate, assicurandomi che il controller orchestrasse il tutto fluidamente.

Un punto di debolezza che riconosco riguarda la gestione degli stati interni nel GameController: man mano che le feature del gioco (meteo, bot, abilità) crescevano, il controller ha rischiato di accumulare troppe responsabilità di orchestrazione. In retrospettiva, avrei potuto scomporre ulteriormente il controllore in micro-gestori più specializzati fin dall'inizio.

Per gli sviluppi futuri, mi piacerebbe ampliare il roster dei Capitani introducendo abilità "passive" (che applicano modificatori perenni alla flotta) o abilità a effetto ritardato (che si attivano dopo n turni), il che richiederebbe un'evoluzione molto interessante della macchina a stati del turno.

Antonellini

Il mio ruolo all'interno del progetto si è concentrato sulla realizzazione degli Automated Players (Bot) e sulla gestione di alcune interfacce grafiche di setup iniziale (selezione bot e capitani) e di collegamento di tutte le altre (viewGUI).

Il principale punto di forza del mio lavoro è la flessibilità del sistema di "intelligenza artificiale". L'aver implementato le difficoltà tramite il pattern Strategy ha disaccoppiato totalmente la logica di mira dal giocatore virtuale. Questo mi ha permesso di scalare facilmente da un bot completamente casuale (Beginner) a uno che apprende dai colpi messi a segno (Pro), fino a uno "truccato" (Sniper), mantenendo il codice pulito.

Come punto di debolezza, ritengo che la logica del livello "Pro" possa essere ulteriormente raffinata. Attualmente l'algoritmo di ricerca della nave (quando viene trovato un pezzo) è efficace, ma non sfrutta concetti probabilistici avanzati (come la "densità di probabilità" basata sulle navi rimaste).

In uno sviluppo futuro del progetto, mi dedicherei all'implementazione di un bot di livello "Specialist" basato su algoritmi più complessi o algoritmi di *Monte Carlo*, capace di calcolare matematicamente le celle con la più alta probabilità di contenere una nave in base agli spazi d'acqua rimanenti.

Solaroli

Sono molto soddisfatto del risultato finale ottenuto dal nostro progetto e, in particolar modo, di come abbiamo lavorato come gruppo, supportandoci a vicenda costantemente per superare gli ostacoli e integrare al meglio le diverse componenti del gioco.

All'interno del team mi sono focalizzato sulla progettazione del Core Model (Grid, Cell, Ship, Fleet), sulla gestione logica e corrispondente interfaccia per il posizionamento della flotta (SetupController e SetupView), e sul sistema di persistenza dei dati (Serializzazione). Inoltre, mi sono occupato dell'implementazione del comparto audio, realizzando la classe SoundManager per la gestione indipendente della colonna sonora e degli effetti sonori.

Essendomi occupato del modello alla base del funzionamento del gioco, ho prestato molta attenzione alla sua robustezza. Ho evitato del tutto l'uso di riferimenti nulli, sfruttando in modo estensivo il costrutto Optional; ho garantito anche l'efficace incapsulamento della gestione dello stato utilizzando copie difensive quando serviva esporlo (List.copyOf()). Questo ha reso il cuore del gioco estremamente solido e immune a manipolazioni esterne accidentali.

Come punto di debolezza, riconosco che l'interfaccia grafica per il posizionamento delle navi risulta funzionale ma un po' statica. L'inserimento tramite click e rotazione tramite pulsante fa il suo dovere, ma non offre il livello di interattività "fluida" che ci si aspetterebbe da un videogioco moderno.

Come sviluppo futuro, mi piacerebbe implementare il sistema di Drag & Drop nell'interfaccia di schieramento, permettendo all'utente di trascinare visivamente le navi sulla griglia. Inoltre, sarebbe interessante rendere le dimensioni della griglia e la composizione della flotta completamente parametrizzabili e customizzabili dall'utente.

Bandini

Sono molto soddisfatto del risultato finale. All'interno del gruppo mi sono occupato principalmente delle meccaniche del Model (gestione turni, meteo dinamico, abilità dei capitani) e della relativa View (animazioni, overlay e widget).

Il punto di forza del mio lavoro è stata la cura architettuale: applicando i pattern Observer e Strategy ho mantenuto il codice estendibile e ho disaccoppiato la logica dei bersagli dall'interfaccia grafica, rispettando il Single Responsibility Principle.

Come punto di debolezza, riconosco che l'utilizzo del pattern Singleton per il WeatherManager non è stata una scelta di design ottimale. Inizialmente sembrava la via più semplice per accedere al meteo in qual, ma mi sono reso conto che ha introdotto uno stato globale nascosto e ha reso i test JUnit molto più macchinosi (costringendomi a forzare il reset manuale dell'istanza). Col senno di poi, avrei preferito gestire il meteo tramite Dependency Injection (passando l'istanza ai costruttori) per mantenere il sistema più disaccoppiato e facilmente testabile.

Per gli sviluppi futuri, si potrebbe espandere il meteo introducendo eventi drastici, come ad esempio tempeste, che possono impedire ai capitani di usare le loro abilità e magari anche cambiare la posizione delle navi.

A. Guida Utente

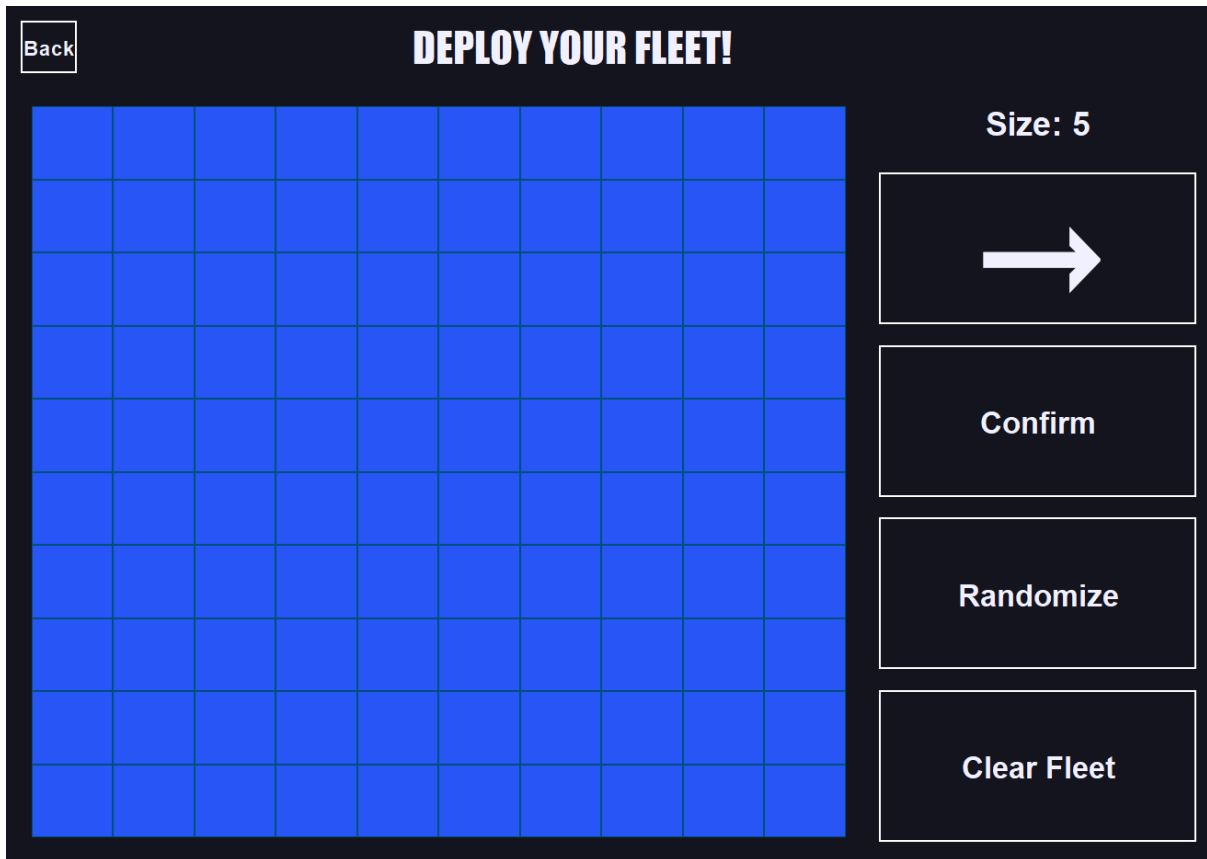
La scelta del capitano e della difficoltà avviene tramite l'apposito menù a tendina

A.1 Schieramento

Nella schermata di posizionamento della flotta l'utente può posizionare le navi cliccando sulla cella desiderata e orientando l'espansione della nave tramite l'apposito pulsante contenente una freccia.

Una volta posizionata la nave è necessario premere il tasto "Conferma" per passare all'inserimento della nave successiva.

E' possibile utilizzare i pulsanti di randomizzazione del posizionamento e di pulizia della griglia in qualsiasi momento dello schieramento. Dopo la conferma dell'ultima nave è possibile premere il pulsante "Start Game", che apparirà al posto del tasto "Confirm", per iniziare la partita



A.2 Svolgimento della Battaglia

Nella schermata principale di gioco le celle possono assumere diverse colorazioni :

- Grigio: rappresenta una cella il cui contenuto è sconosciuto.
- Grigio scuro: rappresentano le navi schierate dall'utente
- Arancione: la cella colpita contiene una nave
- Rosso: la cella colpita contiene una nave affondata
- Blu: La cella colpita non contiene una nave

Mentre tramite l'abilità del sonar le celle potranno assumere i colori:

- Giallo: se è presente almeno una cella che contiene una nave
- Azzurro: se non è presente nessuna nave

Il meteo è contenuto in un widget visivo a schermo che indica la condizione atmosferica attuale. Se l'indicatore segnala "Nebbia", l'utente deve tenere conto che il suo prossimo colpo base potrebbe subire una deviazione casuale nelle celle adiacenti. Il cambiamento della condizione meteorologica viene segnalato tramite un pop-up.

L'abilità del capitano è attivabile tramite l'apposito pulsante. Il tempo di ricarica è scritto su di esso, ed è anche visibile tramite il graduale riempimento del pulsante. Quando il pulsante presenta la scritta "READY" e viene premuto, mostrerà la scritta "ACTIVE" e dovrà essere selezionata la cella su cui si desidera attivarla. In caso di selezione di una cella in cui l'abilità non può essere usata, il tasto ritornerà in modalità "READY".

Le abilità dei capitani funzionano nella seguente maniera:

- Engineer, cura una cella della propria griglia che presenta una nave colpita ma non affondata
- Gunner, genera un colpo 2x2 a partire dalla cella selezionata. L'espansione del colpo avviene in direzione basso-destra. Nel caso venga selezionata una cella che non permette questa espansione, andrà nella direzione opposta (alto-sinistra)
- Sonar, colora un quadrato 3x3 con al centro la cella selezionata in base a cosa contengono

A.3 Salvataggio

E' possibile salvare una partita nella schermata principale di gioco. In caso di inizio di una nuova partita (pulsante "New Game") il salvataggio non sarà più utilizzabile.

B. Esercitazioni di laboratorio

B.0.1 samuele.rocce2@studio.unibo.it

Laboratorio 06 : <https://virtuale.unibo.it/mod/forum/discuss.php?d=206731#p284122>

Laboratorio 07 : <https://virtuale.unibo.it/mod/forum/discuss.php?d=206731#p284122>

Laboratorio 08 : <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p28618>

B.0.2 riccardo.antonellini@studio.unibo.it

Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287255>

Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288211>

Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289557>

B.0.3 enrico.solaroli@studio.unibo.it

Laboratorio 06: <https://virtuale.unibo.it/mod/forum/discuss.php?d=206731&parent=284110>

Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193&parent=284977>

Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921&parent=286197>

Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718&parent=287181>

Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589&parent=288485>

Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617&parent=288919>

Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539&parent=290695>

B.0.4 mattia.bandini3@studio.unibo.it

Laboratorio 06: <https://virtuale.unibo.it/mod/forum/discuss.php?d=206731#p284058>

Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p284838>

Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286089>

Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287148>

Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288422>

Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289547>

Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290662>

Bibliografia

[FSM] - Finite State Machine, modello utilizzato da ProBot per gestire le transizioni logiche comportamentali.

[H/T] - Hunt and Target, variante dell'algoritmo utilizzato da ProBot tramite l'aggiunta di un terzo stato Destroy.